

K0sStreams

Análisis de la idea, propuesta de diferenciación y plan de trabajo

Middlewares Distribuidos — Prof. Lic. R. Alejandro Mansilla

Documento de trabajo interno del grupo (3 integrantes)

21 de septiembre de 2026

Cómo leer este documento: la sección 1 es el resumen de todo en una página. La sección 2 explica los conceptos desde cero con ejemplos de logística. Las secciones 3 a 6 son la opinión, la investigación y la propuesta de mejora. Las secciones 7 a 10 son el plan práctico: horas, cronograma, reparto de tareas y qué preguntarle al profesor.

1. Resumen en una página

La idea es buena y encaja perfecto con la materia, pero tal como está escrita hoy es, en la práctica, una copia a escala de Apache Kafka: segmentos de log, índice disperso, líder por partición, réplicas, quórum y épocas son exactamente las piezas de Kafka. Eso es justo lo que el profesor pidió evitar.

Propuesta de diferenciación: convertir K0sStreams en “**el log que también es cola**”. El mismo tópico se puede leer de dos formas: como historial que se puede releer (como Kafka) y como cola de trabajo con confirmación mensaje por mensaje, reintentos automáticos con espera creciente, depósito de mensajes fallidos (DLQ) y mensajes programados para más tarde (como RabbitMQ o Amazon SQS). Hoy muchas empresas tienen que instalar dos sistemas distintos para tener las dos cosas.

Además, tres mejoras que suman valor y resuelven pedidos del profesor:

- **API HTTP nativa** (con documentación Swagger): cualquiera puede usarlo con un simple curl, Postman o desde cualquier lenguaje. Kafka necesita un componente aparte para esto.
- **Durabilidad verificada:** confirmar al cliente solo después de guardar en disco, y publicar en el repositorio las pruebas de caos que lo demuestran (lección aprendida de fallos reales detectados en NATS en 2025).
- **Tópicos como recurso de Kubernetes (CRD):** se crean con kubectl apply. Esto además cubre el posible requisito de CRD de la cátedra.

Cambios técnicos recomendados: usar HTTP para los clientes y dejar el protocolo binario propio solo para la comunicación interna entre brokers; reemplazar la elección de controlador casera por un **Lease de Kubernetes** (más simple y más seguro); quitar las micro-optimizaciones que no suman nota; corregir afirmaciones incorrectas del documento (por ejemplo, “fallos bizantinos”).

Estimación: unas **225 horas de equipo** usando Claude Code (sin IA serían 350–400). Con 3 personas equivale a **unas 12 horas por semana cada uno** ($\approx 1,5$ h por día hábil + una tarde de fin de semana) desde hoy hasta el 11/11. **Es viable pero ajustado.** La Prueba de Concepto del 07/10 requiere unas 65 horas de equipo: se llega si arrancan esta misma semana.

Dificultad global: 4 de 5

Lo difícil no es programar, es la parte distribuida (replicación y qué pasa cuando se cae un nodo). Por eso el plan deja esa parte para cuando la base ya funcione, y define de antemano qué recortar si el tiempo no alcanza.

2. Qué es K0sStreams, explicado desde cero

Un **middleware orientado a mensajes** es un intermediario entre programas. En vez de que el programa A llame directamente al programa B (y falle si B está caído), A deja un mensaje en el intermediario y B lo retira cuando puede. Es el mismo principio que un centro de distribución: el remitente no necesita que el destinatario esté en su casa en ese momento; deja el paquete en el depósito y la entrega ocurre después.

2.1 Glosario con analogías de logística

Término	Qué es	Analogía logística
Broker	Servidor que recibe, guarda y entrega mensajes.	Un centro de distribución / depósito.
Mensaje (registro)	Un dato que un programa le envía a otro (ej.: "pedido 123 creado").	Un paquete.
Productor / Consumidor	Programa que envía mensajes / programa que los lee.	Remitente / chofer que retira para entregar.
Tópico	Categoría de mensajes con nombre (ej.: pedidos, facturas).	Una línea de reparto (ej.: "Mendoza-Rosario").
Partición	Subdivisión de un tópico para repartir la carga.	Las distintas dársenas de carga de esa línea.
Offset	Número de orden del mensaje dentro de la partición (0, 1, 2...).	El número de guía correlativo.
Log append-only	Archivo donde solo se agrega al final; nunca se borra ni se modifica en el medio.	El libro de registro de entradas: se anota renglón por renglón.
Réplica	Copia de los datos en otro broker.	Copia del remito en otras sucursales.
Líder / Seguidor	El broker que manda sobre una partición / los que copian.	El encargado de turno / los auxiliares.
Quórum	Mayoría (2 de 3) que debe confirmar para dar algo por hecho.	Hace falta la firma de 2 de los 3 responsables.
Ack (acuse)	Confirmación de que algo se hizo.	El remito firmado.
Split-brain	Dos brokers creen ser líder a la vez y aceptan datos distintos.	Dos encargados dando órdenes contradictorias.
Epoch (época)	Número de "mandato" que sube con cada cambio de líder.	Número de turno: con un turno vencido no se puede firmar.
Fencing (cercado)	Rechazar a quien tenga una época vieja.	El encargado del turno anterior ya no puede despachar.
At-least-once	Garantía de que el mensaje llega al menos una vez (puede duplicarse).	El paquete llega seguro; a veces pueden llegar dos avisos.
DLQ (Dead Letter Queue)	Lugar donde van los mensajes que fallaron demasiadas veces.	Depósito de devoluciones / paquetes con problemas.

Término	Qué es	Analogía logística
Visibility timeout	Tiempo que un consumidor tiene para confirmar; si no, el mensaje vuelve a la cola.	Si el chofer no confirma la entrega en 2 h, el paquete se reasigna.
Mensaje diferido	Mensaje que se entrega recién a partir de cierta hora.	Entrega programada para mañana a las 9.

2.2 Cómo viaja un mensaje hoy en K0sStreams (versión propuesta original)

1. El productor envía el mensaje al broker líder de esa partición.
2. El líder lo escribe al final de su archivo de log en disco.
3. El líder se lo reenvía a los otros dos brokers (réplicas).
4. Cuando al menos uno de ellos confirma que lo guardó (2 de 3 = quórum), el líder le responde "OK" al productor.
5. El consumidor pide "dame los mensajes desde el número 150" y el líder se los devuelve.
6. Si el líder se cae, otro broker toma el mando con un número de época más alto, y cualquiera que siga usando la época vieja es rechazado.

Esto es sólido y muy didáctico. El problema, como se explica a continuación, es que es exactamente lo que ya hace Kafka.

3. Mi opinión sobre la propuesta actual

3.1 Lo que está muy bien

- **Encaja con la materia:** es un MOM (unidad de middleware), corre en contenedores y Kubernetes (unidades de cloud native y orquestación) y toca los problemas clásicos de sistemas distribuidos (fallos, particiones de red, consistencia).
- **Está bien especificado:** formato de mensajes, estructura en disco, puertos, manifiestos. Eso ahorra discusiones al programar.
- **No depende de APIs ajenas,** que era justamente lo que les preocupaba con la idea anterior de almacenamiento tipo S3.
- **Es demostrable:** “mato un pod y no se pierde nada” es una demo muy potente para la defensa.

3.2 Problemas que conviene corregir

Problema	Por qué importa	Qué hacer
Solo se usa con un protocolo binario propio por TCP.	Nadie lo puede usar sin escribir un cliente especial; choca con el pedido de “disponible por API para cualquiera”.	API HTTP para clientes; Y gRPC para la comunicación interna entre brokers.
La elección de controlador (“el ID más bajo que ve mayoría”) no es segura.	Con fallas de red asimétricas (A ve a B, B ve a C, pero A no ve a C) dos nodos pueden creerse controlador. Resolverlo bien es implementar un algoritmo de consenso, que es lo más difícil de toda la materia.	Usar un Lease de Kubernetes para elegir al controlador (sección 6). Las épocas y el fencing los siguen implementando ustedes.
Dice que neutraliza “fallos bizantinos”.	Es incorrecto: los fallos bizantinos son nodos que mienten o actúan maliciosamente. Las épocas solo protegen de nodos caídos o aislados. Un profesor de la materia lo va a notar.	Reemplazar por “fallos por caída y particiones de red”.
Micro-optimizaciones (“cero asignaciones”, SocketAsyncEventArgs, pools de buffers).	Cuestan muchas horas y casi no suman nota en un proyecto académico.	Usar System.IO.Pipelines del lado servidor y listo; optimizar solo si sobra tiempo.
Guardar en disco (fsync) mensaje por mensaje.	Es seguro pero muy lento (cientos de mensajes por segundo).	“Group commit”: juntar los mensajes que llegan en unos pocos milisegundos y guardar todos juntos con un solo fsync.

4. Investigación: en qué fallan las colas que ya existen

El objetivo de esta sección es encontrar huecos reales y documentados, no opiniones. Estas son las fallas más relevantes encontradas (septiembre de 2026).

Sistema	Para qué es bueno	Qué le falta o dónde falla
Apache Kafka	Historial de eventos que se puede releer, altísimo volumen.	Nació como log, no como cola. Las “colas” (share groups) recién son estables en la versión 4.2 (2026) y solo con clientes Java. Los mensajes diferidos son todavía una propuesta (KIP-1277). Los reintentos se arman a mano con tópicos extra y se pierde el orden. Para acceder por HTTP hace falta un

Sistema	Para qué es bueno	Qué le falta o dónde falla
		componente separado (REST Proxy) con licencia no Apache. Operación pesada (JVM, mucha memoria).
RabbitMQ	Cola de tareas clásica: ack por mensaje, reintentos, DLQ.	En las colas clásicas el mensaje se borra al confirmarlo: no se puede “volver a leer lo de ayer”. Para eso agregó “Streams”, que es otro tipo de cola con otras reglas: de nuevo, dos modelos separados.
NATS JetStream	Liviano, simple, rápido.	Jepsen (dic. 2025) encontró pérdida de mensajes ya confirmados y split-brain persistente. Una causa: por defecto guardaba en disco cada dos minutos en vez de antes de confirmar.
Apache Pulsar	Une streaming y colas.	Resuelve el problema del modelo doble, pero con una arquitectura de tres sistemas (brokers, BookKeeper y un almacén de metadatos). Muy complejo de operar.
Redpanda	Compatible con Kafka, más eficiente (C++).	Hereda el modelo de Kafka (mismos huecos de cola) y su licencia no es open source estricta (BSL).
Amazon SQS	Cola simple con timeout, DLQ y mensajes diferidos.	No permite releer historial y es un servicio pago de un solo proveedor (no se instala en tu cluster).

4.1 Las tres conclusiones de la investigación

1. El mundo está partido en dos. Los sistemas tipo log (Kafka) son buenos para guardar historial; los sistemas tipo cola (RabbitMQ, SQS) son buenos para repartir trabajo. Confluent, la empresa detrás de Kafka, reconoce que muchas organizaciones mantienen los dos sistemas en paralelo, con el doble de mantenimiento. Kafka está tratando de cerrar esa brecha recién ahora, agregando piezas de a una sobre un diseño que no fue pensado para eso.

2. Reintentar bien es sorprendentemente difícil. En Kafka, si un mensaje falla y querés reintentar en 5 minutos, tenés que crear tópicos extra de reintento, uno por cada tiempo de espera, más un tópico de mensajes muertos, y aceptar que se pierde el orden. Frameworks enteros (Spring Kafka, Wolverine, BabelQueue) existen en parte para tapar este hueco.

3. “Durable” no siempre significa durable. El caso NATS muestra que un sistema popular puede confirmar mensajes que después pierde, y que la única forma de saberlo es probarlo con fallas provocadas. Eso es algo que un proyecto académico puede hacer de manera muy visible.

5. Propuesta de diferenciación: “el log que también es cola”

Frase para la presentación: **K0sStreams es un broker liviano para Kubernetes donde cada tópico es a la vez un historial que se puede releer y una cola de trabajo con reintentos, mensajes programados y DLQ nativos, accesible desde cualquier lenguaje por HTTP y con sus garantías de durabilidad verificadas con pruebas de caos.**

5.1 Pilar 1 — Tópicos de doble modo (el valor agregado principal)

Sobre el mismo tópico conviven dos formas de leer:

- **Modo stream (como Kafka):** “dame todo desde el mensaje 150”. Sirve para auditoría, reconstruir datos o analizar el pasado.
- **Modo cola (como SQS/RabbitMQ):** “dame el próximo trabajo pendiente”. Cada mensaje entregado queda reservado por un tiempo; el consumidor lo confirma (ack), lo devuelve (nack) o, si no responde, vuelve solo a la cola.

Y encima del modo cola, tres funciones nativas que en Kafka hoy no existen de forma simple:

- **Reintentos con espera creciente:** si falla, se reintentará a los 10 s, luego 1 min, luego 5 min (configurable). Sin crear tópicos extra.
- **DLQ automática:** después de N intentos fallidos, el mensaje pasa a un depósito de problemas, con el motivo del error, para revisarlo y reenviarlo.
- **Mensajes programados:** “entregar a partir de mañana a las 9:00”.

Ciclo de vida de un mensaje en modo cola

Estado	Qué significa
LISTO	Disponible para que un consumidor lo tome.
PROGRAMADO	Todavía no llegó su hora (mensaje diferido o esperando un reintento).
EN PROCESO	Un consumidor lo tomó; tiene X segundos para confirmar.
CONFIRMADO	Se procesó bien. Sigue existiendo en el historial (se puede releer), pero ya no se entrega como trabajo.
EN DLQ	Falló demasiadas veces; queda apartado con la causa del error.

La idea técnica clave (para defender ante el profesor)

El estado de la cola (quién tomó qué, cuántos intentos, qué se confirmó) **se guarda también como un log**: cada ack, nack o reintento es un evento más que se escribe y se replica igual que un mensaje común.

Ventaja: no hace falta un segundo mecanismo de replicación. Si el líder se cae, el nuevo líder reconstruye el estado de la cola leyendo ese log. Es la misma idea que usan las bases de datos (event sourcing) y reutiliza el 100% del motor de almacenamiento y replicación que ya planearon.

5.2 Pilar 2 — API HTTP nativa, usable por cualquiera

Cada broker expone una API REST con documentación Swagger generada automáticamente (ASP.NET Core Minimal APIs). No hace falta instalar nada extra ni escribir un cliente especial. Ejemplos:

```
# Crear un t3pico
curl -X POST http://HOST:9090/v1/topics -d '{"name":"pedidos","partitions":3}'

# Publicar (con entrega programada opcional)
curl -X POST http://HOST:9090/v1/topics/pedidos/messages \
  -d '{"key":"cliente-42","value":"...", "deliverAt":"2026-11-01T09:00:00Z"}'

# Modo cola: tomar el pr3ximo trabajo y confirmarlo
curl -X POST http://HOST:9090/v1/topics/pedidos/queues/facturacion/receive
curl -X POST http://HOST:9090/v1/topics/pedidos/queues/facturacion/ack/1532

# Modo stream: releer desde el mensaje 0
curl "http://HOST:9090/v1/topics/pedidos/partitions/0/messages?from=0&max=100"
```

Opcional (si sobra tiempo): recepci3n en vivo con Server-Sent Events, que funciona en cualquier navegador. El protocolo binario de 5 bytes que ya dise1aron **no se tira**: se usa entre brokers (puerto 9091) para la replicaci3n, que es donde el rendimiento importa y donde demuestra lo aprendido sobre protocolos.

5.3 Pilar 3 — Durabilidad verificada, no solo prometida

- **Nunca confirmar antes de guardar en disco.** Para que no sea lento, se usa group commit: se juntan los mensajes de unos milisegundos y se hace un solo guardado.
- **Suite de caos en el repositorio:** scripts que matan pods, cortan la red entre brokers y reinician todo mientras un programa verificador publica y cuenta mensajes. Resultado esperado y publicado en el README: “0 mensajes confirmados perdidos en N ejecuciones”.
- **Recuperaci3n ante archivos da1ados:** al arrancar, cada broker revisa el CRC32 del final de su log, corta lo que est3 roto y lo vuelve a copiar del l3der. (Justamente donde NATS fall3.)

5.4 Complemento — T3picos como recurso de Kubernetes (CRD)

Definir un tipo nuevo de objeto de Kubernetes, “TFopic”, para crear t3picos con un archivo YAML y kubectl apply. No es algo in3dito (Strimzi lo hace para Kafka), as3 que no se presenta como diferenciaci3n sino como integraci3n cloud native, y adem3s cubre el posible requisito de CRD de la c3tedra.

```
apiVersion: k0sstreams.io/v1
kind: Topic
metadata: { name: pedidos }
spec: { partitions: 3, maxRetries: 5, retryBackoff: [10s, 1m, 5m] }
```

5.5 Siendo honestos: ¿es algo totalmente nuevo?

No del todo, y conviene decirlo as3 en la presentaci3n porque suma credibilidad. Kafka est3 incorporando partes de esto reci3n ahora (colas estables en 2026, DLQ en desarrollo, mensajes diferidos en propuesta) y Pulsar lo tiene pero con una arquitectura muy pesada. El aporte de K0sStreams es **la combinaci3n**: las dos sem3nticas dise1adas desde el d3a uno sobre un 3nico log replicado, en un solo binario liviano, accesible por HTTP y con garant3as comprobadas. Para un trabajo acad3mico, identificar un hueco actual con evidencia y resolverlo con un dise1o propio es exactamente el “valor agregado” que pide el profesor.

6. Arquitectura: antes y después

Aspecto	Propuesta original	Propuesta ajustada
Acceso de clientes	Protocolo binario propio por TCP (9090).	API HTTP REST + Swagger (9090). Cualquiera la usa.
Comunicación entre brokers	Protocolo binario (9091).	Igual: protocolo grpc.
Semántica	Solo stream (offsets y grupos).	Stream + cola (ack/nack, timeout, reintentos, DLQ, programados).
Elección de controlador	Algoritmo casero “ID más bajo con mayoría”.	Lease de Kubernetes (coordination.k8s.io). Épocas y fencing propios.
Escritura en disco	Flush por mensaje.	Group commit (lotes de pocos ms).
Rendimiento de red	Cero asignaciones, pools, SocketAsyncEventArgs.	System.IO.Pipelines y nada más.
Creación de tópicos	Opcode CREATE_TOPIC.	Endpoint HTTP + CRD Topic (opcional).
Pruebas	Chaos test de caída de nodo.	Suite de caos automatizada con verificador y resultados publicados.
Uso público	No contemplado.	Repo GitHub + imagen Docker pública + docker compose de 3 nodos.

6.1 Por qué el Lease de Kubernetes

Un Lease es un objeto de Kubernetes que funciona como un “candado con vencimiento”: el broker que lo tiene es el controlador y debe renovarlo cada pocos segundos; si deja de hacerlo, otro lo toma. Kubernetes garantiza que solo uno lo tenga a la vez porque por debajo usa etcd, que ya implementa consenso Raft. Así, la parte más peligrosa del proyecto la resuelve una pieza probada, y ustedes siguen implementando lo interesante: épocas, fencing, replicación y failover.

Si el profesor no acepta depender de Kubernetes para esto, el plan B es usar una librería de Raft para .NET (DotNext.Net.Cluster) en lugar de escribir el consenso desde cero. Escribir Raft propio **no** lo recomiendo con el tiempo disponible.

7. Estimación de dificultad y tiempo

7.1 Supuestos

- 3 integrantes que saben programar pero son nuevos en sistemas distribuidos.
- Cada uno trabaja 6 h/día y cursa 6 h/semana: la disponibilidad real es de noche y fines de semana.
- Uso de Claude Code para generar código, tests y documentación. Reduce mucho el tiempo de escribir código, pero poco el de entender, depurar y probar la parte distribuida.
- Plazos: hoy 21/09, Prueba de Concepto 07/10 (16 días), Proyecto Final 11/11 (7 semanas).

7.2 Horas por módulo

Módulo	Dificultad	Horas equipo	Comentario
Setup: repo, CI, Docker, estructura	Baja	8	Claude Code lo hace casi solo.
Motor de almacenamiento (segmentos, índice, CRC, recuperación)	Media	30	Base de todo. Muy testeable.
API HTTP + Swagger	Baja-media	16	ASP.NET Core lo facilita mucho.
Semántica de cola (ack, timeout, reintentos, DLQ, programados)	Media-alta	32	El diferencial. Muchos casos borde.
Protocolo interno + replicación con quórum	Alta	40	Incluye que un seguidor atrasado se ponga al día.
Coordinación: Lease, épocas, fencing, failover	Muy alta	30	Lo más difícil de depurar.
Despliegue k0s (StatefulSet, PVC, MetalLB)	Media	12	Depende del acceso al cluster de la cátedra.
CRD Topic (opcional)	Media	12	Primer recorte si falta tiempo.
Tests, pruebas de caos, benchmark	Media	26	Clave para la nota final.
Documentación, informes, presentación, demo	Baja	20	No dejarlo para la última semana.
Total		226	≈ 260 con 15% de imprevistos

7.3 ¿Cuántas horas por día?

Cálculo	Resultado
Horas necesarias con margen	≈ 260 horas de equipo
Semanas disponibles	≈ 7,3 (hasta el 11/11)
Horas de equipo por semana	≈ 36
Horas por persona por semana	≈ 12
Traducido a días	1,5 h cada día hábil + 4 a 5 h un día del fin de semana

Cálculo	Resultado
Sin Claude Code	≈ 20 h por persona por semana (no recomendable con su carga)

Veredicto

Viable pero ajustado. La semana de exámenes o una semana laboral pesada puede atrasarlos, por eso el cronograma deja la última semana solo para documentación y demo, y hay una lista de recortes definida de antemano (sección 7.6).

7.4 Qué entra en la Prueba de Concepto (07/10)

Un solo broker funcionando, que ya muestre el diferencial:

- Motor de almacenamiento completo (segmentos, índice, CRC, recuperación al reiniciar).
- API HTTP: crear tópico, publicar, leer desde un offset, Swagger.
- Modo cola básico: receive + ack + visibility timeout (sin reintentos avanzados todavía).
- Imagen Docker y despliegue en un pod de minikube/k0s con volumen persistente.
- README inicial y el documento de propuesta actualizado con la diferenciación.

Estimado: unas 65 horas de equipo en 16 días, es decir ≈ 1,4 h por día por persona.

7.5 Cronograma semanal

Semana	Objetivo
21–27 sep	Repo, estructura, CI. Motor de almacenamiento (escribir y leer segmentos). API HTTP de publicar y leer. Actualizar la propuesta y consultar al profesor (sección 10).
28 sep–4 oct	Índice, CRC, recuperación. Modo cola básico. Docker + minikube/k0s. Tests del almacenamiento.
5–7 oct	Congelar, ensayar la demo y entregar la PoC.
8–14 oct	Reintentos con espera, DLQ, mensajes programados. Protocolo binario interno.
15–21 oct	Replicación con quórum. Seguidor que se pone al día. Replicar también el estado de la cola.
22–28 oct	Lease de controlador, épocas, fencing, failover automático. StatefulSet de 3 nodos + MetalLB.
29 oct–4 nov	Suite de caos y verificador. Corregir errores. CRD Topic si hay tiempo. Benchmark simple.
5–11 nov	Sin funciones nuevas. README, informe final, video de demo, presentación, ensayo.

7.6 Qué recortar si se atrasan (en este orden)

1. CRD Topic.
2. Server-Sent Events (se usa consulta periódica).
3. Varias particiones por tópico (dejar 1 partición por tópico).

4. Mensajes programados (dejar reintentos y DLQ).

Lo que **no** se recorta: replicación con quórum, failover con épocas y el modo cola con ack y DLQ. Eso es el corazón de la nota.

8. Reparto de tareas y cómo usar Claude Code

Integrante	Responsable de	Aprende sobre
Persona A	Motor de almacenamiento + semántica de cola (ack, reintentos, DLQ, programados).	Estructuras en disco, índices, máquinas de estado.
Persona B	API HTTP + protocolo interno + replicación con quórum.	Protocolos, redes, replicación, consistencia.
Persona C	Kubernetes (StatefulSet, PVC, MetalLB, Lease, CRD) + coordinación + suite de caos + CI.	Orquestación, tolerancia a fallos, pruebas de caos.

Todos revisan el código de los demás (pull requests) para que cualquiera pueda explicar cualquier parte en la defensa.

8.1 Reglas para usar bien Claude Code

1. Pedir de a una tarea chica y concreta (un archivo o una función), no “haceme la replicación”.
2. Pedir primero los tests y después el código: los tests son la forma de saber si lo que generó funciona.
3. Dejar en el repo un archivo CLAUDE.md con la arquitectura y las reglas del proyecto, para que Claude Code no invente cosas distintas en cada sesión.
4. Antes de aceptar código, pedirle que lo explique. Si nadie del grupo lo entiende, no se mergea: en la defensa oral lo van a preguntar.
5. La parte distribuida (replicación, épocas) hay que dibujarla en papel o en Miro antes de programarla.

9. Checklist para GitHub y uso público

- Repositorio público con licencia abierta (Apache-2.0 o MIT).
- README con: qué es, por qué es distinto, arranque en 1 comando (docker compose up con 3 brokers), ejemplos con curl y un ejemplo en Python.
- Imagen Docker publicada automáticamente en GitHub Container Registry con GitHub Actions.
- Swagger disponible en /swagger en cada broker.
- Carpeta deploy/ con los manifiestos de Kubernetes y el CRD.
- Carpeta chaos/ con los scripts de caos y los resultados.
- Documento de arquitectura (se puede reutilizar este documento y los diagramas de Miro).
- Si piden un servidor público en internet: agregar una clave de API simple, porque un broker abierto a cualquiera se llena de basura en horas.

10. Qué confirmar con el profesor esta semana

1. ¿La diferenciación “log + cola con reintentos, DLQ y programados, por HTTP” le parece suficiente valor agregado?
2. ¿El CRD es obligatorio para todos los proyectos?
3. “Disponible para cualquiera por API”: ¿alcanza con repo + imagen Docker para que cada uno lo levante, o quiere una instancia pública online?
4. ¿Acepta usar el Lease de Kubernetes para la elección de controlador?
5. ¿Hay acceso al cluster k0s de la cátedra (OpenStack, Cinder, MetalLB) desde ya, o hay que trabajar primero en minikube?

11. Fuentes consultadas

- [Apache Kafka 4.2.0 — anuncio de lanzamiento \(colas / share groups estables\)](#)
- [Confluent — Queues for Kafka GA \(organizaciones con dos sistemas de mensajería\)](#)
- [KIP-932: Queues for Kafka](#)
- [KIP-1277: Support Delayed Message in Kafka \(propuesta\)](#)
- [Apache Kafka PR — DLQ para share groups \(en desarrollo. mayo 2026\)](#)
- [Wolverine — reintentos en Kafka con tópicos escalonados](#)
- [Spring Kafka — reintentos no bloqueantes y DLT](#)
- [Jepsen — análisis de NATS 2.12.1 \(dic. 2025\)](#)
- [Synadia — respuesta al reporte de Jepsen](#)
- [Confluent — REST Proxy como componente separado](#)
- [DZone — REST Proxy bajo Confluent Community License](#)